سیلابس درسی · جلسهٔ ۱ از ۵

چرخهٔ جدید تولید نرم‌افزار با کدنویسی شهودی

از پرامپت‌های (prompt) نامنظم تا مهندسی عامل‌محور (Agentic engineering)

مدت جلسه: ۱۸۰ دقیقه (سه ساعت با دو استراحت) **سطح:** متوسط تا پیشرفته

منبع اصلی: The New SDLC With Vibe Coding — آدی عثمانی و همکاران

فهرست این سیلابس

1. معرفی جلسه
2. اهداف یادگیری
3. پیش‌نیازها و آمادگی فنی
4. مفاهیم کلیدی و واژه‌نامه
5. زمان‌بندی دقیق جلسه
6. کارگاه عملی
7. ارزشیابی و تکلیف
8. منابع و مطالعهٔ تکمیلی

۱. معرفی جلسه

مهم‌ترین تحول سال‌های اخیر در مهندسی نرم‌افزار، یک زبان یا چارچوب تازه نیست؛ **گذار از نوشتن کد به بیان قصد** (intent) است. در این جلسه چارچوب ذهنی کل دوره ساخته می‌شود: طیفی که از «کدنویسی شهودی (Vibe coding)» شروع می‌شود و به «مهندسی عامل‌محور» می‌رسد، مفهوم «مهندسی زمینه (Context engineering)»، «مهار (harness)» (harness) به‌عنوان همهٔ چیزی که مدل را احاطه می‌کند، و بازتعریف نقش توسعه‌دهنده به‌صورت رهبر ارکستر (Conductor) و هماهنگ‌کننده (Orchestrator).

جایگاه این جلسه در دوره

جلسهٔ اول، نقشهٔ کلان است. چهار جلسهٔ بعدی هر کدام یک ضلع از همین نقشه را باز می‌کنند: ابزارها و تعامل‌پذیری (جلسهٔ ۲)، مهارت‌ها (skill) و مهندسی زمینه (context) (جلسهٔ ۳)، امنیت و ارزیابی (evaluation) (جلسهٔ ۴)، و توسعهٔ مبتنی بر مشخصات (SDD) در سطح محصول (جلسهٔ ۵).

۲. اهداف یادگیری

در پایان این جلسه، فراگیر باید بتواند:

#	هدف یادگیری	سطح شناختی
۱	تعریف دقیق «عامل (agent) هوش مصنوعی» را بیان کند و پنج جزء سازندهٔ آن (مدل، ابزار، حافظه، ارکستراسیون، دیپلوی (deploy)) را از هم تفکیک کند.	دانش و فهم
۲	موقعیت یک کار مشخص را روی طیف «کدنویسی شهودی ← کدنویسی ساختارمند ← مهندسی عامل‌محور» تعیین و انتخابش را توجیه کند.	تحلیل
۳	شش نوع زمینه را نام ببرد و برای یک پروژهٔ واقعی، مرز زمینهٔ ایستا (Static) و پویا (dynamic) را طراحی کند.	کاربرد
۴	اجزای «مهار» را فهرست کند و توضیح دهد چرا بیشتر خطاهای عامل، خطای پیکربندی‌اند نه خطای مدل.	تحلیل
۵	تفاوت دو حالت کاری «رهبر ارکستر» و «هماهنگ‌کننده» را تشخیص دهد و برای هر کار، حالت مناسب را انتخاب کند.	کاربرد
۶	«مسئلهٔ ۸۰ درصد (The 80% problem)» را توضیح دهد و راهبردی برای پوشش ۲۰ درصد باقی‌مانده (coverage) پیشنهاد کند.	تحلیل
۷	اقتصاد دو رویکرد را با زبان CapEx/OpEx مقایسه کند و مهندسی زمینه را به‌عنوان اهرم مالی توضیح دهد.	ارزشیابی
۸	یک فایل `AGENTS.md` حداقلی اما کارآمد برای ریپو (repository) خودش بنویسد.	ایجاد

۳. پیش‌نیازها و آمادگی فنی

دانش پیش‌نیاز

- آشنایی کاری با یک زبان برنامه‌نویسی و Git.
- درک کلی از چرخهٔ سنتی تولید نرم‌افزار: نیازمندی، طراحی، پیاده‌سازی، تست، دیپلوی، نگه‌داری.
- تجربهٔ استفاده از یک دستیار کدنویسی (هر سطحی، حتی تکمیل خودکار در ویرایشگر) کافی است؛ دانش یادگیری ماشین لازم نیست.

آمادگی فنی پیش از کلاس

- نصب و ورود به حساب یک عامل کدنویسی خط فرمان (برای نمونه Claude Code، Gemini CLI، Codex CLI یا Cline).
- در دسترس داشتن یک ریپو کوچک و واقعی (ترجیحاً پروژهٔ خود فراگیر) برای تمرین‌ها.
- اطمینان از اجرای تست‌های (test) پروژه با یک فرمان ساده (`npm test` ، `pytest` یا معادل آن).

> **هشدار آمادگی**
>
> اگر فراگیران بدون ابزار (tool) نصب‌شده وارد کلاس شوند، بخش کارگاهی از دست می‌رود. فهرست نصب باید حداقل ۴۸ ساعت پیش از جلسه ارسال شود و در ابتدای کلاس ۵ دقیقه برای رفع اشکال نصب در نظر گرفته شود.

۴. مفاهیم کلیدی و واژه‌نامه

اصطلاح	معادل انگلیسی	تعریف کوتاه
کدنویسی شهودی	`Vibe coding`	توصیف خواسته به زبان طبیعی، پذیرفتن خروجی مدل و بازگرداندن پیام خطا به مدل تا وقتی کار کند؛ بدون راستی‌آزمایی نظام‌مند.
مهندسی عامل‌محور	`Agentic engineering`	همان قدرت مولد، اما درون سامانه‌ای از مشخصات، تست‌ها، ارزیابی‌ها و گاردریل‌ها `(guardrail)` با نظارت انسانی بر معماری.
مهندسی زمینه	`Context engineering`	طراحی آگاهانهٔ اطلاعاتی که عامل در اختیار دارد: دستورها، دانش، حافظه، مثال‌ها، ابزارها و گاردریل‌ها.
زمینهٔ ایستا / پویا	`Static / Dynamic context`	ایستا همیشه بارگذاری می‌شود (هویت `(identity)` عامل)؛ پویا فقط هنگام نیاز فراخوانی می‌شود (مهارت، نتیجهٔ ابزار، بازیابی `(retrieval)` سند).
مهار	`Harness`	هرچه پیرامون مدل قرار می‌گیرد: دستورها، ابزارها، سندباکس، ارکستراسیون، قلاب‌ها `(hook)` و رصدپذیری `(observability)`. **عامل = مدل + مهار**
مدل کارخانه `(Factory model)`	`Factory model`	محصول اصلی کار توسعه‌دهنده دیگر خودِ کد نیست، بلکه سامانه‌ای است که کد را تولید می‌کند.
رهبر ارکستر	`Conductor`	کار هم‌زمان و دستی با عامل در ویرایشگر؛ کنترل دقیق بر هر تغییر.
هماهنگ‌کننده	`Orchestrator`	واگذاری `(handoff)` ناهم‌زمان کار به چند عامل و بازبینی نتیجه؛ کار در سطح انتزاع بالاتر.
مسئلهٔ ۸۰ درصد	`The 80% problem`	عامل ۸۰٪ کد را سریع می‌سازد؛ ۲۰٪ باقی‌مانده (حالت‌های مرزی، اتصال به بخش‌های دیگر،

اصطلاح	معادل انگلیسی	تعریف کوتاه
		جزئیات حساس درستی) شناخت عمیقی از بستر پروژه می‌خواهد.
تست در برابر ارزیابی (Tests vs. Evals)	Tests vs. Evals	تست بخش قطعی (deterministic) را می‌سنجد؛ ارزیابی بخش غیرقطعی (not deterministic) را: مسیر (trajectory) طی‌شده، انتخاب ابزار و کیفیت پاسخ نهایی.

۵. زمان‌بندی دقیق جلسه

00:00 – 00:15

گشایش: چرا این تغییر با تغییرهای قبلی فرق دارد

نظرسنجی سریع از کلاس دربارهٔ میزان استفادهٔ روزانه از عامل‌ها، آمار وضعیت صنعت، و بیان تز اصلی جلسه: گذار از نحو به قصد. اسلاید ۱–۴

00:15 – 00:35

مرور سریع «عامل»: حلقهٔ ادراک ← برنامه ← کنش ← مشاهده

تفاوت عامل با ربات گفت‌وگو، و پنج جزء سازندهٔ هر عامل. این بخش پایهٔ مشترک چهار جلسهٔ بعدی است. اسلاید ۵–۸

00:35 – 01:00

طیف: از کدنویسی شهودی تا مهندسی عامل‌محور

جدول شش‌بُعدی مقایسه، و این نکتهٔ محوری که تمایزدهندهٔ اصلی «نحوهٔ راستی‌آزمایی» است، نه میزان استفاده از هوش مصنوعی. اسلاید ۹–۱۳

01:00 – 01:10

استراحت اول

۱۰ دقیقه.

01:10 – 01:40

مهندسی زمینه: مهارت واقعی

شش نوع زمینه، مرز ایستا/پویا به‌عنوان یک تصمیم معماری، و مهارت‌های عامل به‌عنوان الگوی افشای تدریجی (Progressive disclosure). اسلاید ۱۴–۱۸

01:40 – 02:05

چرخهٔ جدید تولید نرم‌افزار، فاز به فاز

فشرده‌شدن نامتوازن مراحل، تغییر هر فاز، و «مدل کارخانه» به‌عنوان جمع‌بندی ذهنی. اسلاید ۱۹–۲۴

02:05 – 02:25

مهندسی مهار: آنچه مدل را احاطه می‌کند

معادلهٔ «عامل = مدل + مهار»، شش جزء مهار، نقش مهار در هر فاز، و شواهد عددی اثر مهار روی بنچمارک‌ها (benchmark). اسلاید ۲۵–۳۰

02:25 – 02:35

استراحت دوم

۱۰ دقیقه.

02:35 – 02:50

نقش در حال تحول توسعه‌دهنده و مسئلهٔ ۸۰ درصد

دو حالت رهبر ارکستر و هماهنگ‌کننده، مهارت‌های لازم برای حالت دوم، و ماهیت تازهٔ خطاهای هوش مصنوعی. اسلاید ۳۱–۳۵

02:50 – 03:05

هزینه و صرفهٔ اقتصادی توسعه با هوش مصنوعی

بدهی پنهان کدنویسی شهودی، سرمایه‌گذاری اولیهٔ مهندسی عامل‌محور، و مسیریابی (routing) هوشمند مدل. اسلاید ۳۶–۳۹

03:05 – 03:20

کارگاه: نوشتن `AGENTS.md` و جمع‌بندی

تمرین عملی روی ریپو خود فراگیران، سه اصل ماندگار، و پیش‌نمای جلسهٔ دوم. اسلاید ۴۰–۴۴

اگر زمان کم آوردید

بخش «هزینه و صرفهٔ اقتصادی» را می‌توان به ۷ دقیقه فشرده کرد و جدول شش‌بُعدی طیف را به مطالعهٔ جزوه سپرد. اما دو بخش «طیف» و «مهار» هستهٔ جلسه‌اند و حذف‌شدنی نیستند.

۶. کارگاه عملی

تمرین ۱ — جای‌گذاری روی طیف (۱۰ دقیقه، گروه دونفره)

به هر گروه سه سناریو داده می‌شود: یک نمونهٔ اولیهٔ آخر هفته، یک ویژگی جدید روی یک سرویس داخلی پرکاربرد، و یک تغییر در مسیر پردازش پرداخت. هر گروه باید جای هر سناریو را روی طیف مشخص کند و **یک** سازوکار راستی‌آزمایی برای آن تجویز کند.

تمرین ۲ — نوشتن فایل قواعد پروژه (۱۵ دقیقه، انفرادی)

هر فراگیر برای ریپو خودش یک `AGENTS.md` ده‌خطی می‌نویسد که این چهار بخش را داشته باشد:

1. **پشتهٔ فناوری:** زبان، چارچوب، نسخه‌ها و ابزار ساخت.
2. **قراردادها:** سبک کد، ساختار پوشه‌ها، الگوی نام‌گذاری.
3. **قواعد سخت:** کارهایی که عامل هرگز نباید انجام دهد (مثلاً تغییر فایل‌های مهاجرت پایگاه داده یا افزودن وابستگی `(dependency)` جدید بدون اجازه).
4. **گردش کار:** فرمان اجرای تست‌ها، فرمان قالب‌بندی، و انتظار از پیام‌های کامیت `(commit)`.

> **معیار پذیرش تمرین ۲**
> فایل باید چنان باشد که یک توسعه‌دهندهٔ تازه‌وارد هم بتواند فقط با خواندن آن، یک تغییر کوچک را درست انجام دهد. اگر جمله‌ای در فایل برای انسان مبهم است، برای عامل هم مبهم است.

تمرین ۳ — یک قاعده از یک خطا (۵ دقیقه، گفت‌وگوی کلاسی)

فراگیران یک مورد واقعی را نقل می‌کنند که عامل کاری اشتباه انجام داده است؛ سپس آن تجربه را به یک سطر قاعده در `AGENTS.md` ترجمه می‌کنند. این تمرین عادت اصلی دوره را جا می‌اندازد: **هر خطای عامل، یک سطر تازه در مهار است.**

۷. ارزشیابی و تکلیف

مؤلفه	سهم	شرح و معیار نمره‌دهی
مشارکت کلاسی	۱۰٪	کیفیت مشارکت در تمرین جای‌گذاری روی طیف و گفت‌وگوی کلاسی.
فایل `AGENTS.md`	۴۰٪	کامل‌بودن چهار بخش، مشخص‌بودن قواعد سخت، و اجراپذیری فرمان‌های گردش کار. ابهام در جمله‌بندی نمره کم می‌کند.
گزارش کوتاه طیف	۳۰٪	یک صفحه: سه کار واقعی از پروژهٔ خودتان را روی طیف بنشانید و برای هر کدام سازوکار راستی‌آزمایی متناسب پیشنهاد دهید.
یادداشت مطالعه	۲۰٪	خلاصهٔ نیم‌صفحه‌ای از مقالهٔ ترجمه‌شدهٔ جلسه، با تمرکز بر یک ایده که با تجربهٔ شخصی شما هم‌خوانی یا تعارض دارد.

تکلیف تا جلسهٔ بعد

1. `AGENTS.md` نوشته‌شده در کلاس را روی ریپو واقعی خود بکارید و دست‌کم سه کار با عامل انجام دهید.
2. هر بار که عامل کاری کرد که نباید می‌کرد، یک قاعده به فایل بیفزایید و تغییرات را کامیت کنید تا سیر تکامل فایل دیده شود.
3. مقالهٔ ترجمه‌شدهٔ جلسهٔ اول را کامل بخوانید؛ جلسهٔ دوم با پیش‌فرض خواندن آن آغاز می‌شود.

۸. منابع و مطالعهٔ تکمیلی

منبع اصلی جلسه

- مقالهٔ `The New SDLC With Vibe Coding: From ad-hoc prompting to Agentic Engineering` — ترجمهٔ کامل فارسی در همین پوشه در دسترس است.

مطالعهٔ تکمیلی

- آدی عثمانی، «مهندسی عامل‌محور» و «مدل کارخانه» — یادداشت‌های وبلاگی که چارچوب این مقاله از آن‌ها آمده است.
- گوگل، «مقدمه‌ای بر عامل‌ها» (نوامبر ۲۰۲۵) — تشریح عمیق پنج جزء سازندهٔ عامل.

- گزارش METR دربارهٔ اثر واقعی ابزارهای کدنویسی هوش مصنوعی بر سرعت توسعه‌دهندگان باتجربه.

پیش‌نمای جلسهٔ دوم

در جلسهٔ بعد از سطح «مفهوم» به سطح «اتصال» می‌رویم: عامل چگونه با جهان بیرون حرف می‌زند؟ ابزارها، فراخوانی تابع، پروتکل زمینهٔ مدل (MCP) و تعامل عامل با عامل.
